

```
# Basic libraries
import numpy as np # Import numpy for numerical operations
import pandas as pd # Import pandas for data manipulation
import string # Import string module for string operations

# NLP preprocessing
import nltk # Import nltk for natural language processing tasks
from nltk.corpus import stopwords, wordnet # Import stopwords and wordnet from nltk corpus
from nltk.tokenize import word_tokenize # Import word_tokenize for tokenizing text
from nltk.stem import WordNetLemmatizer # Import WordNetLemmatizer for lemmatization

# Vectorization and similarity
from sklearn.feature_extraction.text import TfidfVectorizer # Import TfidfVectorizer for text vectorization
from sklearn.metrics.pairwise import cosine_similarity # Import cosine_similarity for calculating similarity

# Download required NLTK resources
nltk.download('punkt') # Download the 'punkt' tokenizer models
nltk.download('stopwords') # Download the 'stopwords' corpus
nltk.download('wordnet') # Download the 'wordnet' lexical database
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
True
```

```
documents = [
    # Sports
    "The football team won the championship",
    "Cricket players trained hard for the match",
    "The athlete broke the world record",
    "The coach planned a new strategy",

    # Politics
    "The government passed a new law",
    "The election results were announced",
    "Parliament discussed economic reforms",
    "The president addressed the nation",

    # Health
    "Doctors recommend regular exercise",
    "The patient recovered after treatment",
    "A healthy diet improves immunity",
    "Hospitals provide emergency care",

    # Technology
    "Artificial intelligence is transforming industries",
    "The software update improved performance",
    "Cybersecurity protects data from attacks",
    "Cloud computing enables remote work",

    # Mixed for similarity comparison
    "Physicians advise daily workouts",
    "The minister announced new policies",
    "Machine learning improves decision making",
    "The player scored the winning goal",
    "Medical treatment saved the patient",
    "The match was postponed due to rain",
    "Technology helps doctors diagnose diseases",
    "The law was approved by parliament"
]

df = pd.DataFrame({"Document": documents}) # Create a Pandas DataFrame from the documents list
df.head() # Display the first few rows of the DataFrame
```

Document 

- | | Document |
|---|--|
| 0 | The football team won the championship |
| 1 | Cricket players trained hard for the match |
| 2 | The athlete broke the world record |
| 3 | The coach planned a new strategy |
| 4 | The government passed a new law |

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
nltk.download('punkt_tab') # Download the 'punkt_tab' resource for NLTK
stop_words = set(stopwords.words('english')) # Initialize a set of English stopwords
lemmatizer = WordNetLemmatizer() # Initialize the WordNetLemmatizer

def preprocess(text): # Define a preprocessing function
    text = text.lower() # Convert text to lowercase
    text = text.translate(str.maketrans('', '', string.punctuation)) # Remove punctuation
    tokens = word_tokenize(text) # Tokenize the text into words
    tokens = [lemmatizer.lemmatize(word) for word in tokens if word not in stop_words and word.isalpha()] # Lemmatize words
    return tokens # Return the preprocessed tokens

df["Tokens"] = df["Document"].apply(preprocess) # Apply the preprocess function to the 'Document' column to create a new 'Tokens' column
df.head() # Display the first few rows of the DataFrame with the new 'Tokens' column
```

[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!

	Document	Tokens
0	The football team won the championship	[football, team, championship]
1	Cricket players trained hard for the match	[cricket, player, trained, hard, match]
2	The athlete broke the world record	[athlete, broke, world, record]
3	The coach planned a new strategy	[coach, planned, new, strategy]
4	The government passed a new law	[government, passed, new, law]

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
tfidf = TfidfVectorizer(stop_words='english') # Initialize TfidfVectorizer with English stopwords
tfidf_matrix = tfidf.fit_transform(df["Document"]) # Fit and transform the 'Document' column to create a TF-IDF matrix
```

```
cosine_sim = cosine_similarity(tfidf_matrix) # Calculate cosine similarity between all documents based on TF-IDF matrix

cosine_df = pd.DataFrame(cosine_sim, index=df.index, columns=df.index) # Create a DataFrame from the cosine similarity matrix
cosine_df.head() # Display the first few rows of the cosine similarity DataFrame
```

	0	1	2	3	4	5	6	7	8	9	...	14	15	16	17	18	19	20	21	22	23
0	1.0	0.0	0.0	0.000000	0.000000	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.000000
1	0.0	1.0	0.0	0.000000	0.000000	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.21466	0.0	0.000000
2	0.0	0.0	1.0	0.000000	0.000000	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.000000
3	0.0	0.0	0.0	1.000000	0.182569	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.182569	0.0	0.0	0.0	0.000000	0.0	0.000000
4	0.0	0.0	0.0	0.182569	1.000000	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.188252	0.0	0.0	0.0	0.000000	0.0	0.264031

5 rows × 24 columns

```
def jaccard_similarity(doc1, doc2): # Define a function to calculate Jaccard similarity
    set1, set2 = set(doc1), set(doc2) # Convert document tokens to sets
    return len(set1 & set2) / len(set1 | set2) # Calculate and return Jaccard similarity

jaccard_scores = [] # Initialize a list to store Jaccard scores

for i in range(len(df)): # Iterate through each document
    for j in range(i+1, len(df)): # Iterate through subsequent documents to avoid duplicate pairs and self-comparison
        score = jaccard_similarity(df.loc[i, "Tokens"], df.loc[j, "Tokens"]) # Calculate Jaccard similarity for the pair
        jaccard_scores.append((i, j, score)) # Append the document indices and score

jaccard_df = pd.DataFrame(jaccard_scores, columns=["Doc1", "Doc2", "Jaccard"]) # Create a DataFrame from the Jaccard scores
jaccard_df.head() # Display the first few rows of the Jaccard similarity DataFrame
```

	Doc1	Doc2	Jaccard
0	0	1	0.0
1	0	2	0.0
2	0	3	0.0
3	0	4	0.0
4	0	5	0.0

Next steps: [Generate code with jaccard_df](#) [New interactive sheet](#)

```
def wordnet_similarity(word1, word2): # Define a function to calculate WordNet similarity between two words
    syn1 = wordnet.synsets(word1) # Get WordNet synsets for the first word
```

```
syn2 = wordnet.synsets(word2) # Get WordNet synsets for the second word
if not syn1 or not syn2: # If no synsets are found for either word, return 0
    return 0
return syn1[0].wup_similarity(syn2[0]) or 0 # Return Wu-Palmer similarity between the first synsets, or 0 if similarity

def sentence_similarity(sent1, sent2): # Define a function to calculate similarity between two sentences (lists of words)
    scores = [] # Initialize a list to store similarity scores
    for w1 in sent1: # Iterate through words in the first sentence
        for w2 in sent2: # Iterate through words in the second sentence
            scores.append(wordnet_similarity(w1, w2)) # Calculate and append WordNet similarity for each word pair
    return np.mean(scores) if scores else 0 # Return the mean of scores, or 0 if no scores were calculated
```

```
wordnet_results = [] # Initialize a list to store WordNet similarity results

pairs = [(0, 23), (8, 16), (4, 23), (12, 18), (9, 20),
         (1, 21), (2, 3), (6, 19), (10, 22), (14, 15)] # Define specific pairs of document indices for comparison

for i, j in pairs: # Iterate through each defined pair
    sim = sentence_similarity(df.loc[i, "Tokens"], df.loc[j, "Tokens"]) # Calculate sentence similarity using WordNet
    wordnet_results.append((i, j, sim)) # Append the document indices and similarity score

wordnet_df = pd.DataFrame(wordnet_results, columns=["Doc1", "Doc2", "WordNet_Similarity"]) # Create a DataFrame from the W
wordnet_df # Display the WordNet similarity DataFrame
```

	Doc1	Doc2	WordNet_Similarity	
0	0	23	0.277778	
1	8	16	0.275733	
2	4	23	0.335415	
3	12	18	0.255865	
4	9	20	0.350189	
5	1	21	0.222691	
6	2	3	0.247548	
7	6	19	0.219639	
8	10	22	0.218954	
9	14	15	0.182774	

Next steps: [Generate code with wordnet_df](#) [New interactive sheet](#)

```
# Select sentence pairs for comparison
comparison_pairs = [ # Define a list of document index pairs for a comprehensive comparison
    (8, 16), # Doctors recommend regular exercise vs Physicians advise daily workouts
    (4, 23), # The government passed a new law vs The law was approved by parliament
    (12, 18), # Artificial intelligence is transforming industries vs Machine learning improves decision making
    (0, 21), # The football team won the championship vs The match was postponed due to rain (Sports similarity)
    (1, 14), # Cricket players trained hard for the match vs Cybersecurity protects data from attacks (Sports vs Technol
    (9, 20), # The patient recovered after treatment vs Medical treatment saved the patient (Health similarity)
    (5, 17), # The election results were announced vs The minister announced new policies (Politics similarity)
    (10, 22), # A healthy diet improves immunity vs Technology helps doctors diagnose diseases (Health vs Tech)
]
```

```
comparison_results = [] # Initialize a list to store comparison results

for i, j in comparison_pairs: # Iterate through each defined comparison pair
    cosine_score = cosine_sim[i][j] # Retrieve cosine similarity score
    jaccard_score = jaccard_similarity(df.loc[i, "Tokens"], df.loc[j, "Tokens"]) # Calculate Jaccard similarity score
    wordnet_score = sentence_similarity(df.loc[i, "Tokens"], df.loc[j, "Tokens"]) # Calculate WordNet similarity score

    comparison_results.append([ # Append the results for the current pair
        df.loc[i, "Document"], # Original text of the first document
        df.loc[j, "Document"], # Original text of the second document
        round(cosine_score, 3), # Rounded cosine similarity score
        round(jaccard_score, 3), # Rounded Jaccard similarity score
        round(wordnet_score, 3) # Rounded WordNet similarity score
    ])

comparison_df = pd.DataFrame(
    comparison_results,
    columns=["Sentence 1", "Sentence 2", "Cosine", "Jaccard", "WordNet"]
) # Create a DataFrame from the comparison results with specified column names

comparison_df # Display the comparison DataFrame
```

	Sentence 1	Sentence 2	Cosine	Jaccard	WordNet	
0	Doctors recommend regular exercise	Physicians advise daily workouts	0.000	0.000	0.276	
1	The government passed a new law	The law was approved by parliament	0.264	0.167	0.335	
2	Artificial intelligence is transforming indust...	Machine learning improves decision making	0.000	0.000	0.256	
3	The football team won the championship	The match was postponed due to rain	0.000	0.000	0.158	
4	Cricket players trained hard for the match	Cybersecurity protects data from attacks	0.000	0.000	0.123	
5	The patient recovered after treatment	Medical treatment saved the patient	0.518	0.400	0.350	
6	The election results were announced	The minister announced new policies	0.254	0.167	0.268	
7	A healthy diet improves immunity	Technology helps doctors diagnose diseases	0.000	0.000	0.219	

Next steps:

[Generate code with comparison_df](#)[New interactive sheet](#)